

Python Tip: Mutable Defaults

Does your function use `[]` or `{}` as a default value for an argument?

This is dangerous because that default can change over time!

Let's learn why it's dangerous and how to fix it.



A Convenient Default

Imagine we want to make a function that adds an item to a cart. To make it easy to buy a first item, we use an empty list as the default cart.

```
def add_to_cart(item, cart=[]):  
    cart.append(item)  
    return cart  
  
ada_cart = add_to_cart("book")  
ada_cart = add_to_cart("pencil", cart=ada_cart)  
print("Ada:", ada_cart)
```

```
Ada: ['book', 'pencil']
```

Great! Ada can buy her first item without creating an empty cart first. Then she can add a second item to the same cart.

Two Carts Get Mixed Up

But now watch what happens when we use the same function for Grace's shopping. We expect her to get a new cart too.

```
def add_to_cart(item, cart=[]):  
    cart.append(item)  
    return cart  
  
ada_cart = add_to_cart("book")  
grace_cart = add_to_cart("pen")  
  
print("Grace:", grace_cart)  
print("Ada now:", ada_cart)
```

```
Grace: ['book', 'pen']  
Ada now: ['book', 'pen']
```

Grace gets Ada's book, and Ada gets Grace's pen. Both names point to the same cart!

Why Does This Happen?

Python creates a default value once when it defines the function. It does not create a new default for each call.

A list is **mutable**, which means it can be changed over time. Each call changes the same default list.

By contrast, strings, integers, and `None` are **immutable**. They cannot be changed in place, so they are safe defaults.

Watch Your Defaults

Avoid mutable defaults. Use immutable defaults instead.

```
# Dangerous: mutable defaults
def add(item, items=[]): ...
def configure(settings={}): ...
def collect(values=set()): ...

# Safe: immutable defaults
def greet(name="friend"): ...
def retry(count=3): ...
def add_safe(item, items=None): ...
```

Lists, dictionaries, sets, and other mutable objects can all cause this bug.

Build a Safe Cart

So how do we fix our cart function? Use `None` as the default, then create a new list when the function is called.

```
def add_to_cart(item, cart=None):  
    if cart is None:  
        cart = []  
  
    cart.append(item)  
    return cart  
  
print("Ada:", add_to_cart("book"))  
print("Grace:", add_to_cart("pen"))
```

```
Ada: ['book']  
Grace: ['pen']
```

It is a little annoying, but this is a well-known wart in Python. Each call now gets a new cart.

Wrap-Up

Never use a mutable object like `[]`, `{}`, or `set()` as a default argument.

Python creates the default once, so later calls can reuse changed data.

Use `None`, then create a new object inside the function.

Follow me for more tips.

- Shep4